Bring Wall-E into MuJoCo

Import steps for the existing Onshape export

No new CAD export is required for the identified mass issues. Use the existing URDF and mesh assets as the geometry source, then prepare corrected inertial properties and a MuJoCo-native model.

Verified installation and import status

On 10 October 2026, system Python (python3) successfully imported **MuJoCo 3.7.0** and mujoco.viewer. The project's .venv could not import MuJoCo. Viewer-module availability was checked; a graphical viewer window was not tested.

Current source: **Assembly-Wall-E/assembly_1/urdf/assembly_1.urdf**. Its exported mass totals approximately **238.833 g**; this is not a verified physical robot mass.

Direct loading failed. A second in-memory import with visual meshes retained and fixed-body fusion enabled failed at the same component:

```
inertia must have positive eigenvalues
Element: 0603_resistor_004_2
```

The individual component inertias must be repaired or aggregated before compilation. Enabling fixed-body fusion alone does not bypass this validation error.

1. Prepare the corrected import

• Preserve the original export. Work on a derived copy and retain every mesh's scale, orientation, and placement.

• Collapse fixed components into three moving bodies: chassis, left wheel, and right wheel. Preserve the two wheel hinge axes and joint relationships.

• Use the confirmed 153 g battery-and-holder group under the shared uniform-density assumption, 4.5 g per installed screw, 40 g per wheel, and 29 g per complete motor. Reconcile the populated-board total and remaining components.

• Recompute center of mass and inertia as well as mass. For combined bodies, transform component inertias into a common frame and apply the parallel-axis theorem. Do not retain the one-cell inertia for a three-cell mesh.

• Resolve package:// mesh references to available assets. Keep detailed meshes as visual geometry; add deliberate collision shapes later.

Checkpoint: valid aggregate inertias, correct geometry transforms, consistent units (kg, m, kg·m²), and a documented mass total. Check against the complete robot's measured mass when supplied.

Load, view, then verify

2. Save the prepared model as MJCF

MJCF is MuJoCo's native XML format. A corrected URDF can be compiled with MjModel.from_xml_path and saved using mj_saveLastXML, or its geometry and corrected rigid-body properties can be written directly into MJCF.

Proposed output: **simulation/models/wall_e.xml**. This file has not been created yet. A first geometry preview can be kept static while physical parameters are completed; it is not a validated dynamic model.

3. Open the prepared model

From the project root, once wall_e.xml exists:

```
python3 -m mujoco.viewer --mjcf=simulation/models/wall_e.xml
```

This uses the existing system-Python installation. Inspect scale, mesh alignment, body orientation, wheel spacing, and joint axes before relying on simulated motion.

4. Integrate MuJoCo into the project environment

For later use alongside the existing Gymnasium/PPO workflow, install the same verified version into .venv:

```
.venv/bin/python -m pip install "mujoco==3.7.0"
```

Verify the installed version:

```
.venv/bin/python -c "import mujoco; print(mujoco.__version__)"
```

These are instructions, not completed installation steps. Record the selected dependency version in the project configuration and experiment artifacts when integration is implemented.

5. Add and check physical behavior incrementally

After the required measurements are available, add a floating chassis, floor, simple wheel/chassis collision geometry, and measured motor behavior. Check passive falling, command polarity, contact, saturation, and timestep convergence before adding PPO.

The existing export has no collision definitions. A successful mesh preview does not establish correct ground contact, motor response, or physical balancing. Keep wheel position/speed out of deployable policy inputs unless actually measured or reliably estimated; the TT motors have no encoders.

Official references

MuJoCo Python: installation and standalone viewer

MuJoCo modeling: URDF import and MJCF conversion

Related project guide: docs/MUJOCO_STEPS.pdf. Rebuild this PDF with python3 docs/builders/build_mujoco_import_steps.py.